



Impact of Environment Mismatch on ML Model Performance & Production Readiness Measures

Name	USN
Harsh Agrawal	1RV23AI036
Abhishek Biradar	1RV23AI027
Abhilash Maiya	1RV23AI004
Aanish Khan	1RV23AI002

Why this issue matters?

- **Machine learning models do not operate independently — they rely on:**
 - Python version
 - ML libraries (NumPy, Pandas, Scikit-learn, PyTorch, etc.)
 - Preprocessing utilities
 - Underlying mathematical engines (BLAS, MKL, CUDA)
- **Any change in these components can alter:**
 - Model behavior
 - Prediction values
 - Data transformations
 - Reproducibility



What Happened?

- The MedInsure claim-approval model was **trained in Python 3.8**, but the staging server used **Python 3.10**.
- Core ML libraries (NumPy, Pandas, Scikit-learn) were different between environments.
- As a result:
 - **Prediction outputs in staging did not match development outputs.**
 - QA results were invalidated.
 - The model could not be certified for production.

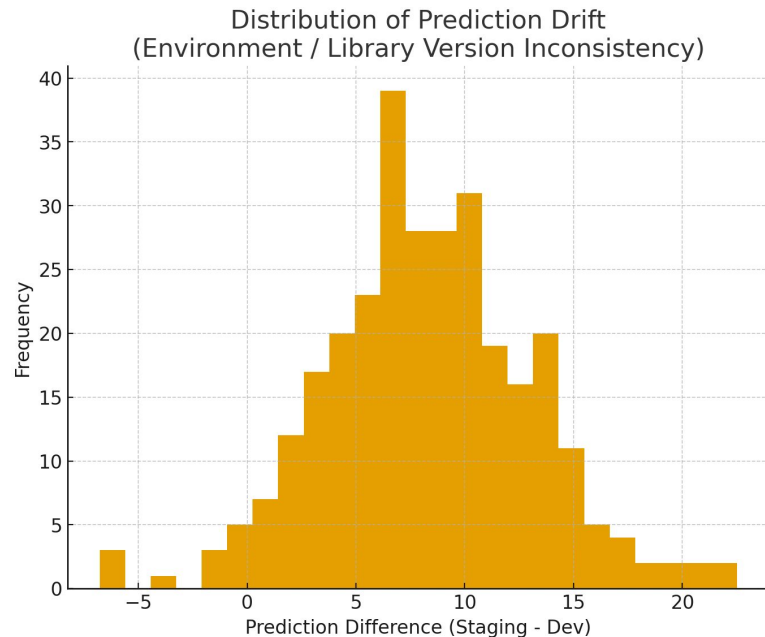
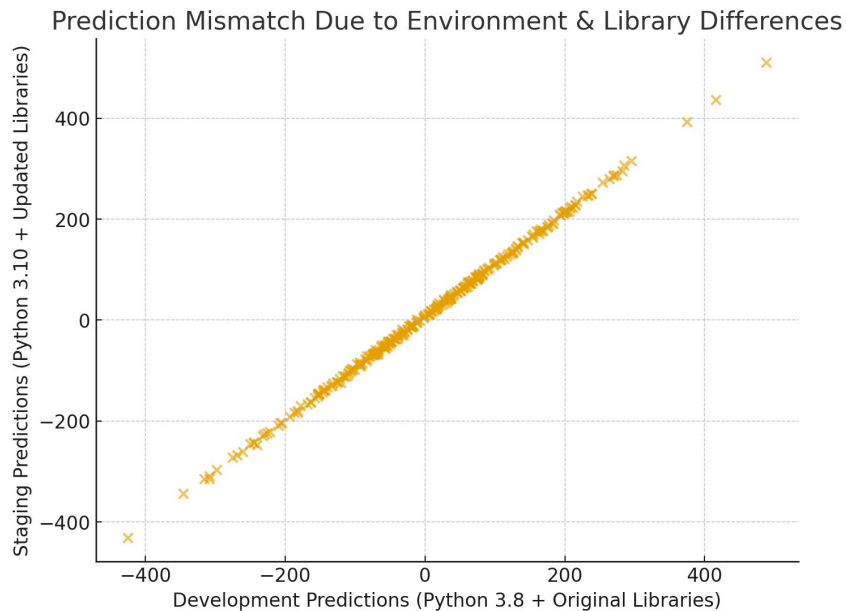
Why this is critical?

In a health-insurance context:

- Even small prediction changes can mean **wrong claim approvals or rejections**.

Prediction outputs in staging did not match development outputs.

Even small prediction changes can mean wrong claim approvals or rejections.





1. Algorithm Output Drift

- Scikit-learn modifies internal logic across versions.
- Tree-based models & linear models can produce **different predictions** even with the same inputs.
- This explains why MedInsure saw prediction mismatch in staging.

2. Preprocessing Pipeline Differences

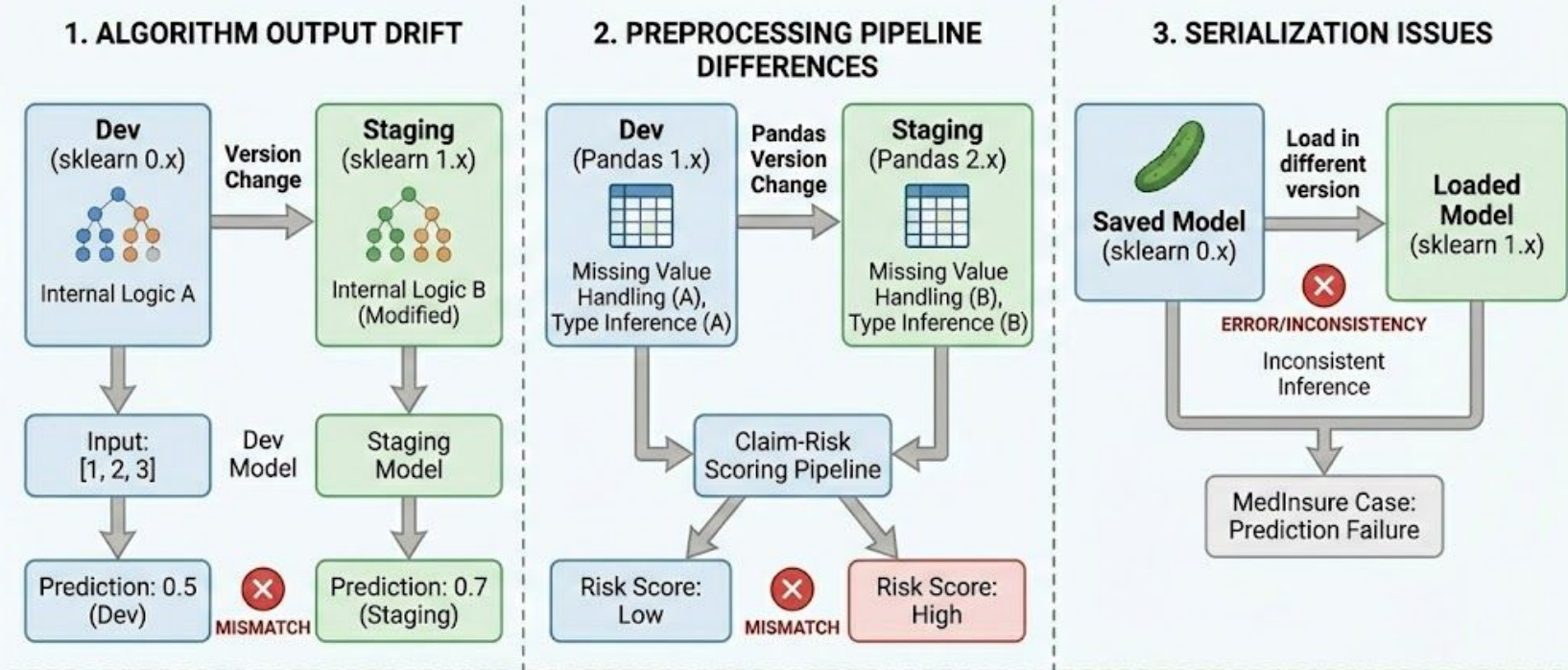
- Pandas version changes can alter:
 - Missing value handling
 - Data type inference
 - Column ordering
- This impacts the model's claim-risk scoring pipeline.



3. Serialization Issues

- A model saved in sklearn 0.x (dev) may load differently in sklearn 1.x (staging).
- MedInsure faced inconsistent inference due to this.

Sources of Model Drift & Prediction Mismatch



Key Drivers of Prediction Mismatch Across Environments



Technical Impact

- Inference results differed → staging environment showed unpredictable model scores.
- Internal validation metrics lost meaning.
- Debugging became extremely difficult.

Business Impact

- Incorrect risk scoring may:
 - Approve fraudulent claims
 - Reject legitimate low-risk claims
- Customer dissatisfaction and financial loss.
- Regulatory audit failure because results are **not reproducible**.

How MedInsure Will Address the Environment Issues

To resolve the prediction inconsistencies and environment mismatches found during pre-production, MedInsure will implement **three core fixes**:

- **1. Standardizing the Runtime Environment**

Using Docker to ensure Development, Staging, and Production all run the *same* Python and library versions.

- **2. Locking All Dependencies**

Using pip-tools, Poetry, or Conda to freeze package versions and prevent accidental upgrades that caused prediction drift.

- **3. Tracking Models & Environments for Reproducibility**

Using MLflow and DVC to record training environments, datasets, and model metadata so results can be reliably reproduced and audited

Why MedInsure needs Docker

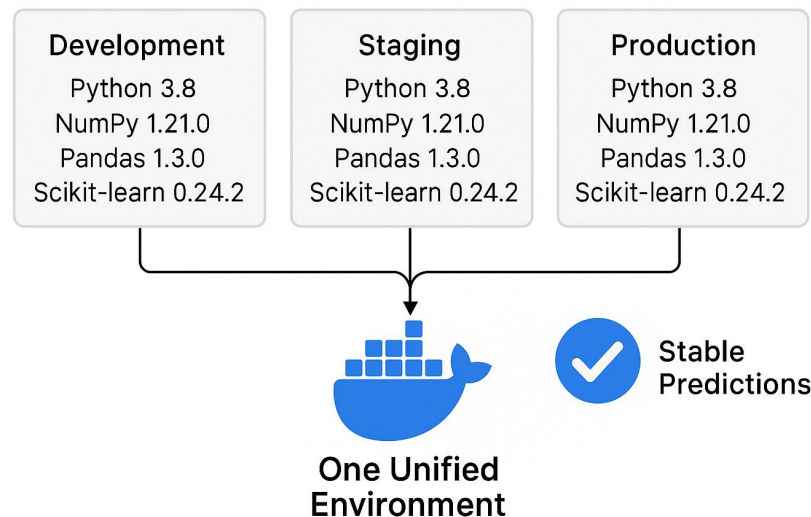
- Ensures Dev, Staging, and Production run **identical Python + library versions**.
- Eliminates runtime mismatch that caused prediction drift in the case study.

MedInsure Should:

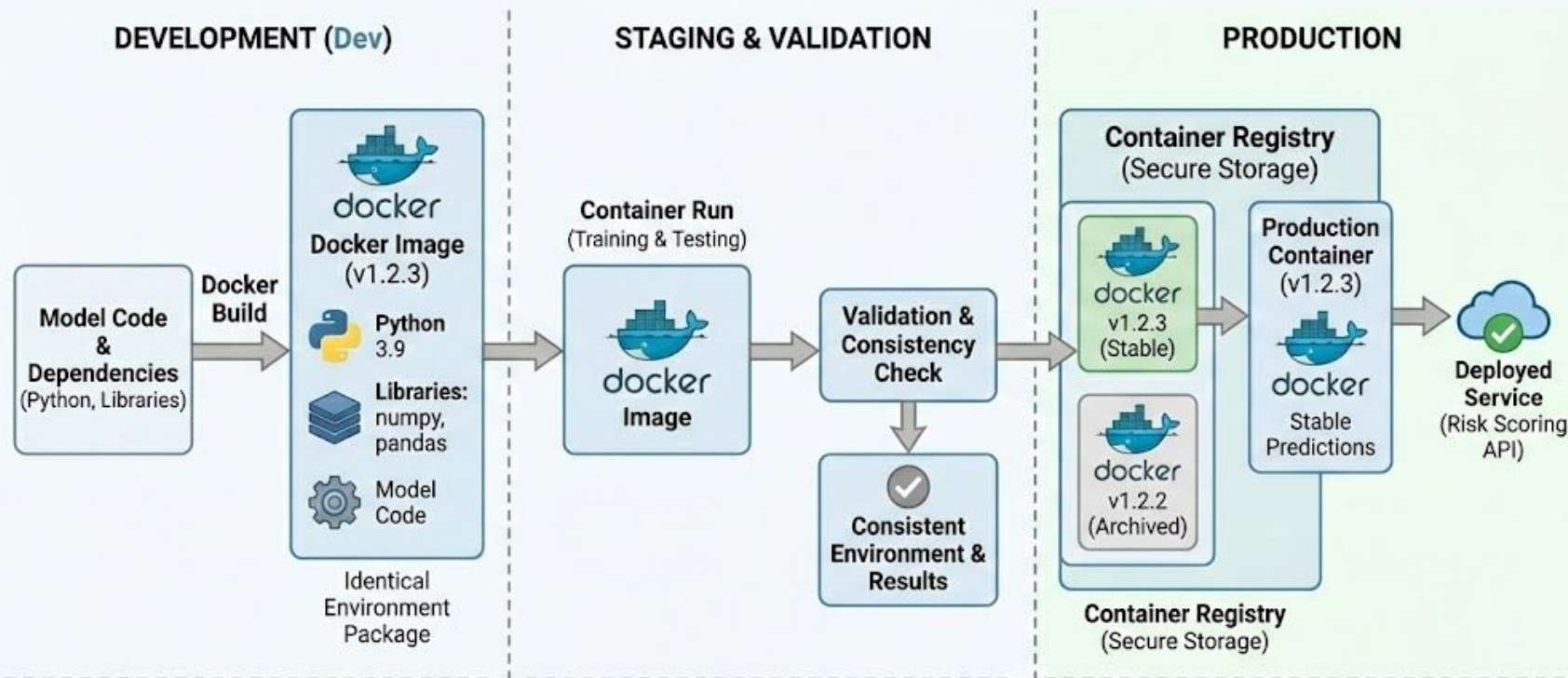
- Train the model inside a Docker image.
- Run validation inside the same image.
- Deploy that exact container to production.
- Store image versions in a secure registry.

Results

- Consistent risk-scoring outcomes.
- Stable, predictable predictions across environments.



MedInsure Dockerized Model Workflow & Consistency



Ensures Identical Dev, Staging, and Production for Consistent Risk-Scoring Outcomes

Tools MedInsure Should Use:

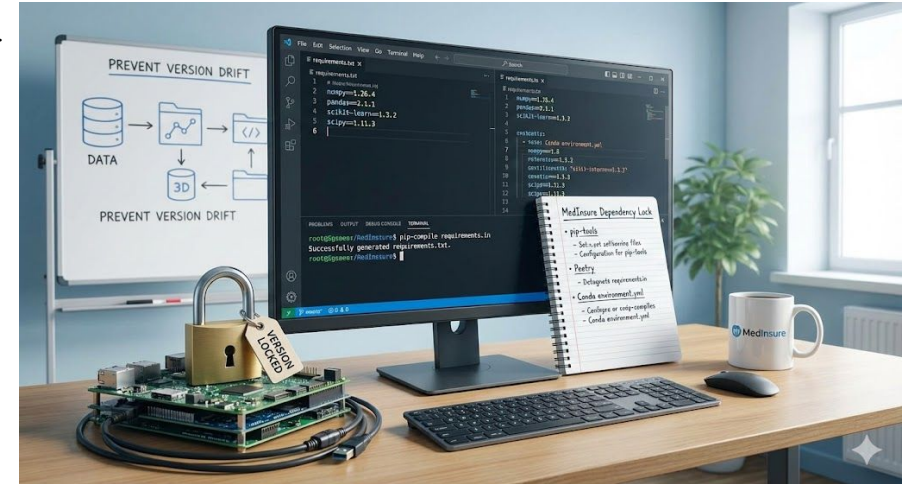
- **pip-tools** → create version-locked dependency files.(command example: pip freeze > reqs.txt)
- **Conda environment.yml** → freeze Python + library versions.

Benefits for MedInsure:

- Prevents accidental upgrades of Pandas, NumPy, sklearn.
- Ensures claim-scoring logic remains identical over time.
- Makes every model run auditable for regulators.

Example:

`numpy==1.26.4` instead of `numpy>=1.18`





Use MLflow for:

- Logging training environment (Python version, library versions).
- Saving trained model artifacts.
- Tracking model lineage for audits.
- Registering stable production models.

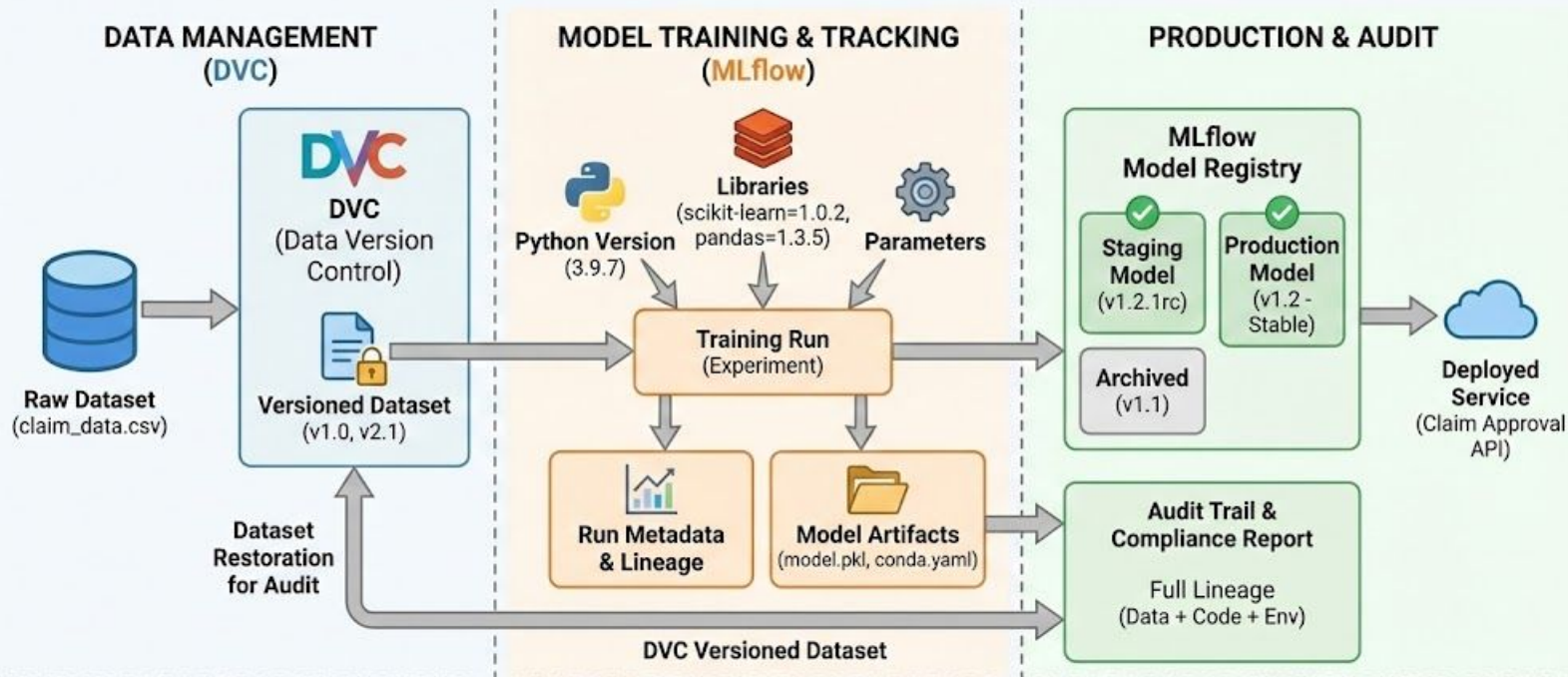
Use DVC for:

- Versioning the dataset used for training the claim-approval model.
- Guaranteeing the exact dataset can be restored if required by auditors.

Why this matters for MedInsure:

- Insurance models require **full reproducibility** and **compliance**.
- Auditors asked MedInsure for training metadata → they couldn't provide it.
- MLflow + DVC solve this permanently.

MedInsure MLflow & DVC Audit Workflow



Ensuring Reproducibility, Compliance, and Full Lineage for Regulatory Audits



Is the MedInsure model ready for production?
No — not until environment consistency is enforced.

Why?

- Dev vs staging mismatch caused prediction drift.
- Model could not be validated or audited.
- Wrong claim approvals pose financial and legal risk.

Required Fixes Before Deployment:

- Build and deploy the model using **a single Docker environment**.
- Freeze Python + library versions using lock files.
- Use MLflow to track model + environment metadata.
- Use DVC to track exact training datasets.
- Validate the model *inside the same environment* that will run in production.



Outcome After Fixes

- Consistent predictions
- Auditable model lineage
- Safe, compliant, production-ready claim-approval system



RV College of
Engineering®

Go, change the world

THANK YOU